

Python for Web Development

Lesson 2: HTTP Basics and Flask Introduction

HTTP Basics and Flask Introduction - Study Notes

Key Concepts

- HTTP Request/Response Cycle
- Flask Application Structure
- Route Decorators and View Functions
- HTTP Methods (GET, POST, etc.)
- Request and Response Objects in Flask

Summary

This lesson covers the fundamentals of how web applications communicate using the HTTP protocol and introduces Flask as a lightweight Python web framework. The HTTP request/response cycle forms the foundation of web communication, where clients send requests to servers and receive responses. Flask simplifies web development by providing a minimal yet powerful framework that handles the complexities of HTTP while allowing developers to focus on application logic. The lesson demonstrates how to build a basic Flask application that responds to HTTP requests, illustrating the direct mapping between HTTP concepts and Flask implementation.

Summary

The HTTP protocol defines how clients (like web browsers) and servers communicate over the internet. When a client sends an HTTP request, it includes a method (GET, POST, etc.), a URL/path, headers, and optionally a body. The server processes this request and returns an HTTP response containing a status code, headers, and optionally a body. Flask is a Python microframework that abstracts these HTTP mechanics into Python functions and decorators. By using route decorators (`@app.route`), developers map URL paths to Python functions that handle incoming requests and generate appropriate responses. This lesson demonstrates building a minimal Flask application that responds to HTTP GET requests, showing how the framework translates HTTP concepts into Python code.

Important Points

- HTTP is a stateless protocol where each request-response pair is independent
- Common HTTP methods: GET (retrieve data), POST (submit data), PUT/PATCH (update), DELETE (remove)
- HTTP status codes: 2xx (success), 3xx (redirection), 4xx (client errors), 5xx (server errors)
- Flask uses decorators to map URLs to view functions (`@app.route('/path')`)
- Flask automatically handles HTTP details like parsing requests and formatting responses
- The request object in Flask provides access to incoming data (args, form, json, etc.)
- The response object or return values from view functions generate HTTP responses
- Debug mode in Flask enables auto-reloading and detailed error pages during development
- Flask applications are created by instantiating the Flask class (`app = Flask(__name__)`)

Examples

****Example 1: Basic Flask Application****

```
```python
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
 return '<h1>Hello, World!</h1>'
if __name__ == '__main__':
 app.run(debug=True)
```
```

***Explanation:** This creates a minimal Flask application. The `@app.route('/')` decorator maps the root URL (`/`) to the `home()` function. When a client sends a GET request to `/`, Flask calls this function and returns its string value as the HTTP response body with a 200 OK status. Running with `debug=True` enables automatic reloading on code changes and provides detailed error pages.

****Example 2: Handling Query Parameters****

```
```python
from flask import Flask, request
app = Flask(__name__)
@app.route('/greet')
def greet():
 name = request.args.get('name', 'Guest')
 return f'<h1>Hello, {name}!</h1>'
if __name__ == '__main__':
 app.run()
```
```

***Explanation:** This demonstrates handling query parameters. When a client requests `/greet?name=Alice`, `request.args.get('name', 'Guest')` retrieves the `name` parameter from the query string, defaulting to `'Guest'` if not provided. The function returns a personalized greeting. A request to `/greet` without parameters would return "Hello, Guest!".

Quick Review

1. What HTTP method is typically used to retrieve data from a server, and which Flask decorator maps a URL to a function?
2. How does Flask access query parameters from a URL like `/search?q=python&category=tutorial`?
3. What does setting `debug=True` in `app.run()` accomplish during development?
4. What HTTP status code range indicates client errors, and give one common example?
5. How does Flask determine the HTTP status code when a view function returns a string?

Further Reading

- HTTP/2 and HTTP/3 protocol improvements
- RESTful API design principles
- Flask extensions (Flask-RESTful, Flask-SQLAlchemy, Flask-WTF)

- Client-side technologies that interact with HTTP (AJAX, Fetch API)
- Deployment options for Flask applications (Gunicorn, uWSGI, Docker)
- HTTP security considerations (HTTPS, CORS, CSRF protection)
- Advanced Flask patterns (application factories, blueprints, blueprints)

HTTP Basics and Flask Introduction - Study Notes

Key Concepts

- HTTP Request/Response Cycle
- Flask Application Structure
- Route Decorators and View Functions
- HTTP Methods (GET, POST, etc.)
- Request and Response Objects in Flask

Summary

This lesson covers the fundamentals of how web applications communicate using the HTTP protocol and introduces Flask as a lightweight Python web framework. The HTTP request/response cycle forms the foundation of web communication, where clients send requests to servers and receive responses. Flask simplifies web development by providing a minimal yet powerful framework that handles the complexities of HTTP while allowing developers to focus on application logic. The lesson demonstrates how to build a basic Flask application that responds to HTTP requests, illustrating the direct mapping between HTTP concepts and Flask implementation.

The HTTP protocol defines how clients (like web browsers) and servers communicate over the internet. When a client sends an HTTP request, it includes a method (GET, POST, etc.), a URL/path, headers, and optionally a body. The server processes this request and returns an HTTP response containing a status code, headers, and optionally a body. Flask is a Python microframework that abstracts these HTTP mechanics into Python functions and decorators. By using route decorators (@app.route), developers map URL paths to Python functions that handle incoming requests and generate appropriate responses. This lesson demonstrates building a minimal Flask application that responds to HTTP GET requests, showing how the framework translates HTTP concepts into Python code.

Important Points

- HTTP is a stateless protocol where each request-response pair is independent
- Common HTTP methods: GET (retrieve data), POST (submit data), PUT/PATCH (update), DELETE (remove)
- HTTP status codes: 2xx (success), 3xx (redirection), 4xx (client errors), 5xx (server errors)
- Flask uses decorators to map URLs to view functions (@app.route('/path'))
- Flask automatically handles HTTP details like parsing requests and formatting responses
- The request object in Flask provides access to incoming data (args, form, json, etc.)
- The response object or return values from view functions generate HTTP responses
- Debug mode in Flask enables auto-reloading and detailed error pages during development
- Flask applications are created by instantiating the Flask class (app = Flask(__name__))

Examples

****Example 1: Basic Flask Application****

```
```python
```

```

from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
 return '<h1>Hello, World!</h1>'
if __name__ == '__main__':
 app.run(debug=True)

```

**\*Explanation:** This creates a minimal Flask application. The `@app.route('/')` decorator maps the root URL (`/`) to the `home()` function. When a client sends a GET request to `/`, Flask calls this function and returns its string value as the HTTP response body with a 200 OK status. Running with `debug=True` enables automatic reloading on code changes and provides detailed error pages.

**\*\*Example 2: Handling Query Parameters\*\***

```

python
from flask import Flask, request
app = Flask(__name__)
@app.route('/greet')
def greet():
 name = request.args.get('name', 'Guest')
 return f'<h1>Hello, {name}!</h1>'
if __name__ == '__main__':
 app.run()

```

**\*Explanation:** This demonstrates handling query parameters. When a client requests `/greet?name=Alice`, `request.args.get('name', 'Guest')` retrieves the `'name'` parameter from the query string, defaulting to `'Guest'` if not provided. The function returns a personalized greeting. A request to `/greet` without parameters would return "

